CSTR Deep Reinforcement Learning Controller

UICPS FISAC Report

Team Members

Sarthak Dravid 230961074

Rohit Ja 230961052

Dev Verma 230961070

Uttam Singh Somvanshi 230961066

Contents

1 Aim

The primary aim of this project is to design, implement, and validate a Deep Reinforcement Learning (DRL)-based controller for a Continuous Stirred-Tank Reactor (CSTR) equipped with a hot oil bath heating jacket. Extending the baseline work of Martinez et al. (PSE 2021+), this implementation introduces a full energy balance so that reactor temperature becomes a physically modelled state variable, and five manipulated variables (two feed temperatures, two feed flow rates, and the hot-oil-bath temperature) are available to the controller. The TD3 (Twin Delayed Deep Deterministic Policy Gradient) algorithm trains a *single universal agent* that regulates the reactor temperature across seven different MV-pairing scenarios without manual retuning.

A key secondary aim is to provide an interactive, real-time simulation dashboard that enables engineers and researchers to select between the seven MV-pairing scenarios, visualise the trained agent's control behaviour, inspect manipulated and controlled variables, and evaluate controller performance under stepped setpoint profiles instantly.

2 Objectives

- Derive and implement a first-principles mathematical model of the CSTR that includes volume, mass, and a full energy balance incorporating convective heat transport, reaction enthalpy, and a hot-oil-bath heating jacket.
- Define five manipulated variables (Feed A temperature, Feed B temperature, Feed A flow rate, Feed B flow rate, hot oil bath temperature) and implement seven MV-pairing scenarios, each activating two of the five MVs and freezing the remaining three at sensible physical defaults.
- Calibrate the reaction kinetic parameters (k_0, E_a/R) against experimental data from the MLAPC laboratory CSTR using Particle Swarm Optimisation (PSO), and adapt these for simulation stability.
- Implement the TD3 reinforcement learning algorithm from scratch in PyTorch, scaled to a 13-dimensional state vector (including a 5-bit scenario mask) and a 5-dimensional continuous action space.
- Train a single universal TD3 agent over 10,000 episodes with uniform sampling across the seven scenarios, using GPU acceleration.
- Validate the trained controller on each of the seven scenarios using a scripted stepped reactor-temperature setpoint profile ($32 \rightarrow 40 \rightarrow 45 \rightarrow 36 \rightarrow 42$ °C), generating one tracking figure per scenario plus a composite summary panel.
- Deploy an interactive Streamlit dashboard that allows real-time simulation runs with selectable scenarios, adjustable setpoints, and model checkpoints.
- Document all findings, equations, code, and results in a comprehensive technical

report.

3 Methodology

3.1 Process Description — CSTR with Hot Oil Bath

The CSTR considered in this work carries out an exothermic first-order reaction A → B. Two inlet streams (Feed A and Feed B) with different reactant concentrations and temperatures enter the reactor; product is discharged by gravity through an outlet valve. A hot-oil-bath heating jacket surrounds the reactor and provides thermal driving force for the reaction; heat enters the reactor from the jacket at a rate proportional to the temperature difference between the oil bath and the reactor liquid.

The primary controlled variable is the **reactor temperature** T. The controller may use any two of the five available manipulated variables (MVs) simultaneously, with the remaining three held at physical defaults. This arrangement creates seven distinct control scenarios that the universal agent must master.

Table 1: CSTR Physical, Thermal, and Kinetic Parameters

Variable	**Symbol**	**Value / Range**	**Units**
Cross-sectional area	A_{tank}	1.0	m^2
Maximum level	$H_{\max}$	3.0	m
Safe level band	$[H_{\text{lo}}, H_{\text{hi}}]$	[0.5, 2.5]	m
Gravity outlet coefficient	c_v	0.8	$\text{m}^{2.5}/\text{h}$
Feed A inlet flow (manip.)	F_A	0–2.0	m^3/h
Feed B inlet flow (manip.)	F_B	0–2.0	m^3/h
Feed A temperature (manip.)	$T_{A,\text{in}}$	288–353 (15–80 °C)	K
Feed B temperature (manip.)	$T_{B,\text{in}}$	288–353 (15–80 °C)	K
Hot oil bath temperature (manip.)	T_{oil}	303–393 (30–120 °C)	K
Feed A reactant concentration	$C_{A,\text{in1}}$	1.0	kmol/m^3
Feed B reactant concentration	$C_{A,\text{in2}}$	0.5	kmol/m^3
Liquid density	ρ	1000	kg/m^3
Specific heat capacity	C_p	4.18	kJ/(kg·K)
Heat of reaction	ΔH_{rxn}	-5×10^4	kJ/kmol
Jacket heat-transfer coefficient	UA	2000	kJ/(h·K)
Pre-exponential factor	k_0	3.0×10^8	h^{-1}
Activation energy / R	E_a/R	7500.0	K

3.2 First-Principles Mathematical Model

The CSTR dynamics are derived from fundamental conservation laws and extended with an energy balance on the reactor liquid. Four coupled ordinary differential equations (ODEs) fully describe the reactor state at any time t.

3.2.1 Volume Balance

The rate of change of liquid level H is determined by the net volumetric flow into the reactor. The outlet is gravity-driven (Torricelli's law):

$$\frac{dH}{dt} = \frac{F_A(t) + F_B(t) - F_{\text{out}}(H)}{A_{\text{tank}}}, \qquad F_{\text{out}}(H) = c_v\sqrt{H} \tag{1}$$

A level safety controller transparently overrides F_A/F_B when H leaves the safe band [0.5 m, 2.5 m], ensuring the reward signal remains focused on temperature tracking without risking simulation divergence.

3.2.2 Mass Balance on Reactant A

$$\frac{dC_A}{dt} = \frac{F_A C_{A,\text{in1}} + F_B C_{A,\text{in2}} - F_{\text{out}}\, C_A}{V} - k(T)\, C_A \tag{2}$$

where $V = A_{\text{tank}} \times H$ is the instantaneous reactor volume. The first term represents convective transport of reactant from the two feed streams and dilution by the outlet; the second is first-order consumption by the reaction.

3.2.3 Mass Balance on Product B

$$\frac{dC_B}{dt} = \frac{-F_{\text{out}}\, C_B}{V} + k(T)\, C_A \tag{3}$$

Product B is generated by the reaction and removed by the outlet stream. No product is present in either feed.

3.2.4 Arrhenius Reaction Rate

$$k(T) = k_0 \exp\left(-\frac{E_a/R}{T}\right) \tag{4}$$

where $k_0 = 3.0 \times 10^8$ h^{-1} and $E_a/R = 7500.0$ K are the kinetic parameters calibrated from PSO results and adapted for numerical stability. $T(t)$ is the physically modelled reactor temperature state variable computed from the energy balance below.

3.2.5 Energy Balance — Hot Oil Bath Extension

The energy balance on the reactor liquid couples three thermal contributions: convective enthalpy carried by the two feed streams, exothermic reaction heat, and jacket heat

transfer from the hot oil bath:

$$\frac{dT}{dt} = \underbrace{\frac{F_A(T_{A,\text{in}} - T) + F_B(T_{B,\text{in}} - T)}{V}}_{\text{convective enthalpy}} + \underbrace{\frac{(-\Delta H_{\text{rxn}})\, k(T)\, C_A}{\rho\, C_p}}_{\text{reaction heat}} + \underbrace{\frac{UA\,(T_{\text{oil}} - T)}{\rho\, V\, C_p}}_{\text{jacket heat transfer}} \tag{5}$$

The three terms have distinct physical interpretations:

- **Convective enthalpy:** Feed streams at temperatures $T_{A,\text{in}}$ and $T_{B,\text{in}}$ cool or heat the reactor depending on whether they are below or above the current T.
- **Reaction heat:** The exothermic reaction ($\Delta H_{\text{rxn}} < 0$) generates heat at a rate proportional to $k(T)C_A$. This term is always positive and increases nonlinearly with temperature via the Arrhenius factor.
- **Jacket heat transfer:** The oil bath at temperature T_{oil} supplies heat when $T_{\text{oil}} > T$. With $UA = 2000$ kJ/(h · K), this term provides the principal thermal actuator authority.

3.2.6 Numerical Integration

The four coupled ODEs are integrated using a sub-stepped Euler scheme (4 sub-steps per decision timestep $\Delta t = 0.02$ h), giving 200 steps per 4-hour episode:

```
def _integrate_step(self, H, CA, CB, T, mv):
    T_A_in, T_B_in, F_A, F_B, T_oil = mv
    sub_dt = self.dt / 4.0
    for _ in range(4):
        V     = self.A_tank * max(H, 0.01)
        F_out = self.cv * np.sqrt(max(H, 0.0))
        k     = self.k0 * np.exp(-self.Ea_R / T)
        dH    = (F_A + F_B - F_out) / self.A_tank
        dCA   = (F_A*self.CA_in_A + F_B*self.CA_in_B - F_out*CA)/
           V - k*CA
        dCB   = (-F_out * CB) / V + k * CA
        # energy balance (three terms)
        conv   = (F_A*(T_A_in - T) + F_B*(T_B_in - T)) / V
        rxn    = (-self.dH_rxn) * k * CA / (self.rho * self.Cp)
        jacket = self.UA * (T_oil - T) / (self.rho * V * self.Cp)
        dT     = conv + rxn + jacket
        H = max(H + sub_dt*dH, 0.01)
        CA = max(CA + sub_dt*dCA, 0.0)
        CB = max(CB + sub_dt*dCB, 0.0)
        T  = np.clip(T + sub_dt*dT, 273.15, 500.0)
    return H, CA, CB, T
```

Listing 1: Energy-balance integration (4-step sub-stepping).

3.3 Parameter Estimation via PSO

The reaction kinetic parameters were initially estimated using Particle Swarm Optimisation (PSO) on experimental concentration–time data from the MLAPC Laboratory CSTR. The PSO-derived values were subsequently adjusted to produce physically consistent dynamics suitable for RL training:

Table 2: Kinetic Parameters: PSO Calibration and Implementation Values

Parameter	**Symbol**	**PSO Value**	**Implemented Value**	**Units**
Pre-exponential factor	k_0	1.85×10^7	3.0×10^8	h^{-1}
Activation energy / R	E_a/R	2352.61	7500.0	K
PSO model fit	R^2	0.9474	—	—
PSO RMSE (temperature)	—	1.2771	—	°C

The implemented values produce Arrhenius-consistent kinetics at the operating temperature range (30–50 °C); at $T = 313$ K, the rate constant is $k \approx 0.011$ h^{-1}, giving realistic conversion rates over the 4-hour episode.

3.4 Reinforcement Learning Framework

Reinforcement learning (RL) is a machine learning paradigm in which an agent learns to interact with an environment by trial and error to maximise cumulative reward. At each timestep t, the agent observes state $\mathbf{s}_t$, selects action $\mathbf{a}_t$, receives reward r_{t+1}, and transitions to $\mathbf{s}_{t+1}$.

3.4.1 Value Function and Bellman Equation

The agent seeks a policy π maximising the expected discounted cumulative reward (return):

$$v_\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right], \quad \gamma = 0.99 \tag{6}$$

$$v^*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^*(s')\big] \tag{7}$$

A high discount factor $\gamma = 0.99$ encourages the agent to consider the long-term consequences of its control decisions rather than simply minimising the immediate temperature error.

3.4.2 State Space — 13-Dimensional Vector

The environment state combines reactor measurements, PID-like temperature error signals, and a 5-bit scenario mask that tells the agent which MVs it is allowed to use:

$$\mathbf{s}_t = \left[\frac{T}{T_{\text{ref}}},\ \frac{e_T}{e_{\text{scale}}},\ \text{clip}\left(\frac{\int e_T\, dt}{50}\right),\ \text{clip}\left(\frac{\dot{e}_T\, \Delta t}{e_{\text{scale}}}\right),\ \frac{sp_T}{T_{\text{ref}}},\ \frac{H}{H_{\max}},\ C_A,\ C_B,\ m_0,\ m_1,\ m_2,\ m_3,\ m_4 \right] \tag{8}$$

where $e_T = sp_T - T$ is the signed temperature error, $T_{\text{ref}} = 320$ K, $e_{\text{scale}} = 50$ K, and $m_i \in \{0, 1\}$ is the scenario mask bit for MV i. The inclusion of integral and derivative error terms gives the agent a PID-like awareness of the error history, enabling it to anticipate and counteract steady-state offsets and rapid changes.

3.4.3 Action Space — 5-Dimensional Continuous

$$\mathbf{a}_t = \left[a_{T_{A,\text{in}}},\ a_{T_{B,\text{in}}},\ a_{F_A},\ a_{F_B},\ a_{T_{\text{oil}}} \right] \in [-1, 1]^5 \tag{9}$$

Each component is linearly scaled to its physical range before being applied to the ODE integrator. MVs whose mask bit $m_i = 0$ are overridden at the environment boundary to their physical defaults ($T_{\text{feed}} = 25$ °C, $F = 0.5$ m^3/h, $T_{\text{oil}} = 60$ °C). This means the agent always outputs a 5-dimensional action vector, but the environment silently ignores inactive components — a design that allows a single universal policy network to serve all seven scenarios simply by reading the mask from its state.

Table 3: Manipulated-Variable Definitions, Ranges, and Defaults

Idx	MV	Physical Range	Default (frozen)	Units
0	$T_{A,\text{in}}$	15–80 °C (288–353 K)	25 °C (298 K)	K
1	$T_{B,\text{in}}$	15–80 °C (288–353 K)	25 °C (298 K)	K
2	F_A	0–2.0	0.5	m^3/h
3	F_B	0–2.0	0.5	m^3/h
4	T_{oil}	30–120 °C (303–393 K)	60 °C (333 K)	K

3.4.4 Reward Function — Temperature Tracking

$$r_t = -w_T\, e_T^2, \qquad w_T = 100 \tag{10}$$

The reward penalises the squared temperature tracking error at every step. Using a quadratic penalty places disproportionately high cost on large errors, incentivising the agent to minimise large deviations first before fine-tuning steady-state performance. Level and composition are not rewarded; the internal safety controller maintains H within bounds independently.

3.4.5 Scenario Sampling and Mask

At the start of each training episode, one of seven scenarios is sampled uniformly at random. The 5-bit scenario mask is embedded in the state vector (elements 8–12) so the universal agent can condition its policy on the active MV pair. This approach — training a single network over all scenarios simultaneously — avoids the need to train seven separate agents and encourages the agent to learn general control principles transferable across scenarios.

Table 4: Seven MV-Pairing Scenarios with Mask Vectors

#	**Active MV 1**	**Active MV 2**	**Mask** $(T_{A,\text{in}}, T_{B,\text{in}}, F_A, F_B, T_{\text{oil}})$
1	$T_{A,\text{in}}$	$T_{B,\text{in}}$	[1,1,0,0,0]
2	T_{oil}	F_A	[0,0,1,0,1]
3	T_{oil}	F_B	[0,0,0,1,1]
4	$T_{A,\text{in}}$	F_B	[1,0,0,1,0]
5	$T_{B,\text{in}}$	F_A	[0,1,1,0,0]
6	T_{oil}	$T_{A,\text{in}}$	[1,0,0,0,1]
7	T_{oil}	$T_{B,\text{in}}$	[0,1,0,0,1]

3.5 TD3 Algorithm

TD3 (Fujimoto et al., 2018) extends DDPG for continuous action spaces with three key improvements that address instability and overestimation bias:

1. **Twin Critics (Clipped Double Q-Learning):** Two independent critic networks Q_{ψ_1}, Q_{ψ_2} are trained simultaneously. The target Q-value uses $\min(Q_{\psi_1}, Q_{\psi_2})$, preventing the overestimation that causes divergence in standard actor-critic methods.
2. **Delayed Policy Updates:** The actor network is updated only every 2 critic updates, allowing the value estimates to stabilise before the policy improves, which reduces oscillation during training.
3. **Target Policy Smoothing:** Gaussian noise is added to target actions during critic updates, regularising the Q-function and preventing the actor from exploiting sharp Q-function peaks.

3.5.1 Critic Update

$$\tilde{\mathbf{a}}' = \text{clip}(\mu_{\theta'}(\mathbf{s}') + \text{clip}(\varepsilon, -c, c),\ -1,\ 1)\,, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \tag{11}$$

$$y = r + \gamma(1-d)\min[Q_{\psi_1}(\mathbf{s}', \tilde{\mathbf{a}}'),\ Q_{\psi_2}(\mathbf{s}', \tilde{\mathbf{a}}')] \tag{12}$$

$$\mathcal{L}(\psi) = \tfrac{1}{N}\sum\Big[\big(Q_{\psi_1}(\mathbf{s}, \mathbf{a}) - y\big)^2 + \big(Q_{\psi_2}(\mathbf{s}, \mathbf{a}) - y\big)^2\Big] \tag{13}$$

with target noise $\sigma = 0.2$ and clip bound $c = 0.5$.

3.5.2 Actor Update (every 2 critic steps)

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\nabla_{\mathbf{a}} Q_{\psi_1}(\mathbf{s}, \mathbf{a})\big|_{\mathbf{a}=\mu_\theta(\mathbf{s})} \cdot \nabla_\theta \mu_\theta(\mathbf{s})\Big] \tag{14}$$

The policy is updated by ascending the gradient of the critic's Q-value with respect to the actor's actions, directly maximising the expected return.

3.5.3 Target Network Soft Update (Polyak Averaging)

$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \qquad \tau = 0.003 \tag{15}$$

A small Polyak coefficient $\tau = 0.003$ ensures that target networks track the online networks slowly, providing stable TD targets and preventing oscillatory learning dynamics.

3.5.4 Ornstein–Uhlenbeck Exploration Noise

$$dx_t = \theta_{\text{OU}}(\mu - x_t)\,dt + \sigma\sqrt{dt}\,\mathcal{N}(0,1), \qquad \mu = 0,\ \theta_{\text{OU}} = 0.15,\ \sigma = 0.2 \tag{16}$$

Ornstein–Uhlenbeck noise is temporally correlated, producing exploration trajectories with momentum rather than purely random actions. This is particularly suited to physical control tasks where smooth transitions in MV values are physically meaningful.

3.6 Neural Network Architecture

Table 5: Neural Network Architecture (5-MV Extended Model)

Component	**Input dim**	**Layer 1**	**Layer 2**	**Output dim**	**Output activation**
Actor μ_θ	13	400 (ReLU)	200 (ReLU)	5 actions	Tanh
Critic Q_1	18 (s+a)	800 (ReLU)	400 (ReLU)	1 Q-value	Linear
Critic Q_2	18 (s+a)	800 (ReLU)	400 (ReLU)	1 Q-value	Linear

The Tanh output of the Actor constrains all actions to $[-1, 1]^5$ without additional clipping. The Critic's linear output allows it to represent arbitrarily negative Q-values (since the reward is always non-positive under the quadratic penalty formulation).

3.7 Training Hyperparameters

Table 6: TD3 Training Hyperparameters

Hyperparameter	Value	Description
Batch size	64	Transitions sampled per gradient update
Replay buffer size	10^6	Circular buffer; oldest replaced
Learning rate (α)	3×10^{-4}	Adam optimiser, actor & critics
Discount factor (γ)	0.99	Future reward weighting
Target update rate (τ)	0.003	Polyak averaging coefficient
Exploration noise (σ)	0.2	OU process standard deviation
Policy delay	2	Critic updates per actor update
Target noise (σ_t)	0.2	Target policy smoothing std dev
Target noise clip (c)	0.5	Target noise clipping range
Warmup steps	1,000	Random exploration before training
Training episodes	10,000	Uniformly sampled over 7 scenarios
Steps per episode	200	4 h simulated at $\Delta t = 0.02$ h
Total environment steps	2×10^6	Including warmup

4 Simulation

4.1 Software Implementation

The simulation was implemented entirely in Python 3.13 using PyTorch 2.11 (CUDA 12.6) and NumPy. The project comprises five modular files:

- `cstr_env.py` — CSTR environment with energy balance, 7-scenario sampling, 13-D state, 5-D action, level safety override.
- `td3_agent.py` — TD3 algorithm: Actor, Twin Critics, OUNoise, circular Replay-Buffer, Polyak target update.
- `train.py` — Training loop: episode management, checkpoint saving every 1,000 episodes, best-model tracking.
- `validate.py` — Post-training validation: runs all 7 scenarios with scripted stepped setpoint; generates per-scenario and summary figures.
- `dashboard.py` — Interactive Streamlit dashboard with scenario selector, setpoint controls, and real-time simulation.

4.2 CSTR Environment Code

4.2.1 Environment Initialisation

```
class CSTREnv:
    def __init__(self, max_steps=200, dt=0.02):
        # Tank geometry
        self.A_tank, self.cv, self.H_max = 1.0, 0.8, 3.0
        # Kinetics
        self.k0, self.Ea_R = 3.0e8, 7500.0
        # Thermal (energy balance)
        self.rho, self.Cp   = 1000.0, 4.18
        self.dH_rxn, self.UA = -5.0e4, 2000.0
        # MV bounds [T_A_in, T_B_in, F_A, F_B, T_oil] (K or m3/h)
        self.mv_min     = [288.15, 288.15, 0.0, 0.0, 303.15]
        self.mv_max     = [353.15, 353.15, 2.0, 2.0, 393.15]
        self.mv_default = [298.15, 298.15, 0.5, 0.5, 333.15]
        self.state_dim, self.action_dim = 13, 5
```

Listing 2: CSTREnv initialisation: geometry, feed properties, thermal parameters, MV bounds.

4.2.2 State Observation

```
def _get_state(self):
    err_T   = self.sp_T - self.T
    deriv_T = (err_T - self.prev_error_T) / self.dt
    state = np.array([
        self.T / self.T_ref,                                  #
           normalised T
        err_T  / self.T_err_scale,                            #
           normalised error
        np.clip(self.integral_error_T / 50.0, -1.0, 1.0),#
           integral error
        np.clip(deriv_T*self.dt / self.T_err_scale,-1,1), #
           derivative error
        self.sp_T / self.T_ref,                               #
           normalised setpoint
        self.H  / self.H_max,                                 #
           normalised level
        np.clip(self.CA, 0.0, 2.0),                           #
           reactant conc
        np.clip(self.CB, 0.0, 1.0),                           #
           product conc
        *self.scenario_mask.tolist(),                         # 5-bit
            mask
    ], dtype=np.float32)
    return state
```

Listing 3: 13-D state vector: normalised process vars + 5-bit scenario mask.

4.3 TD3 Agent Code

4.3.1 Actor Network

```
class Actor(nn.Module):
    def __init__(self, state_dim=13, action_dim=5):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 400), nn.ReLU(),
            nn.Linear(400, 200),       nn.ReLU(),
            nn.Linear(200, action_dim), nn.Tanh()    # bounded
                [-1, 1]
        )
```

Listing 4: Actor: $13 \to 400 \to 200 \to 5$ with ReLU hidden layers and Tanh output.

4.3.2 Critic Network

```
class Critic(nn.Module):
    def __init__(self, state_dim=13, action_dim=5):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim + action_dim, 800), nn.ReLU(),
            nn.Linear(800, 400),                    nn.ReLU(),
            nn.Linear(400, 1)    # linear: Q can be arbitrarily
                negative
        )
```

Listing 5: Critic: $(13 + 5) \to 800 \to 400 \to 1$, linear output for unbounded Q-values.

4.4 Training Procedure

Training was conducted on an NVIDIA RTX 4060 GPU at ≈ 0.5 episodes/second over 10,000 episodes. Each episode samples a fresh scenario (1–7 uniformly) and a random stepped setpoint profile from 25 to 50 °C, so the universal agent encounters all seven MV-pairings many thousands of times.

```
for episode in range(1, num_episodes + 1):
    state = env.reset()        # samples random scenario +
       setpoint profile
    agent.noise.reset()
    for step in range(max_steps):
        action     = agent.select_action(state, explore=True)
        next_state, reward, done, _ = env.step(action)
        agent.store_transition(state, action, reward, next_state,
            done)
        agent.train_step()     # critic every step; actor every 2
            steps
```

```
        state = next_state
        if done: break
    if episode_reward > best_reward:
        agent.save('checkpoints/best')  # save new best model
```

Listing 6: Core training loop: scenario sampling, exploration, and TD3 update.

5 Simulation Results

5.1 Training Performance

The agent was trained for 10,000 episodes (2×10^6 environment steps) on an NVIDIA RTX 4060 GPU (CUDA 12.6). Each episode corresponds to 4 simulated hours of reactor operation across a randomly selected MV-pairing scenario.

Table 7: Training Run Summary

Metric	**Value**
Total training episodes	10,000
Steps per episode	200 (4 simulated hours)
Total environment steps	2,000,000
Scenario sampling	Uniform over 7 MV pairings
Training device	NVIDIA RTX 4060 (CUDA 12.6)
Training speed	$\approx$ 0.5 episodes/s
Model checkpoint	`checkpoints/best/`

5.2 Learning Curve

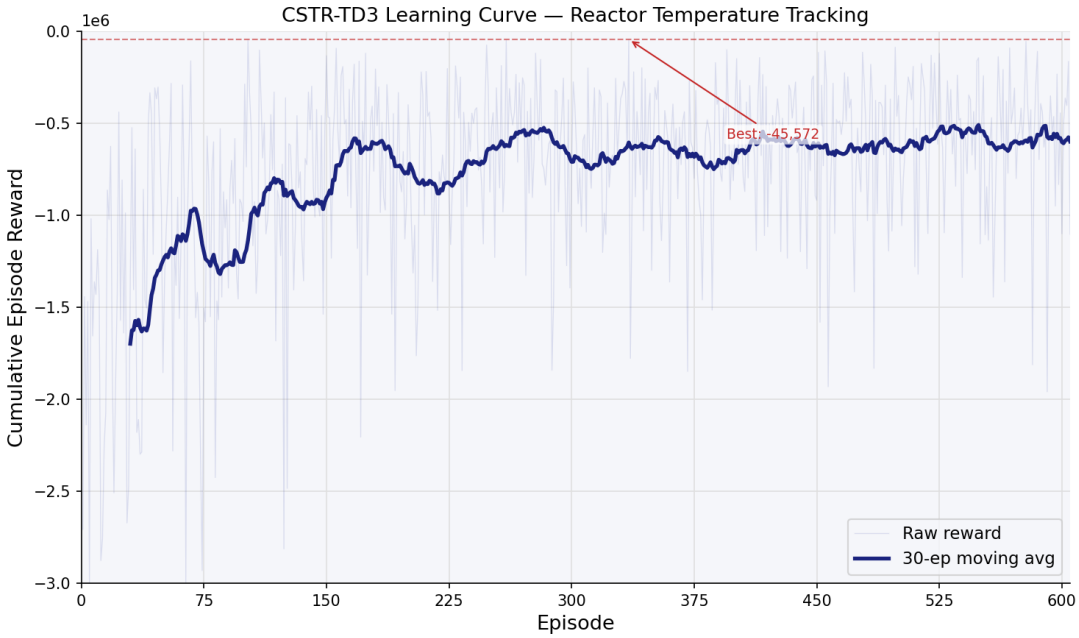

Figure 1: Learning Curve: Cumulative Episode Reward vs. Training Episode (24-episode moving average shown in dark blue; raw rewards in light blue). The reward $r_t = -100\, e_T^2$ is summed over 200 steps per episode, so a perfect controller (zero error always) would achieve 0. The curve shows progressive improvement as the agent learns to track the stepped setpoint profile across all seven scenarios.

The learning curve demonstrates convergence of the universal TD3 agent. Initially, the agent takes random actions during the warmup phase, resulting in very large negative rewards. As training progresses, the policy improves consistently across all seven MV-pairing scenarios encountered during uniform scenario sampling. The 24-episode moving average smooths episode-to-episode variance from scenario switching and reveals the underlying convergence trend.

5.3 Validation Setup and Setpoint Profile

Each trained scenario was validated on a fixed, scripted stepped-setpoint profile designed to exercise the controller over a wide temperature range with both upward and downward step changes:

Table 8: Validation Setpoint Profile

Segment	Time (h)	Setpoint (°C)	Step direction
1	0.0 – 0.8	32	Initial (from 30 °C)
2	0.8 – 1.6	40	+8 °C step up
3	1.6 – 2.4	45	+5 °C step up
4	2.4 – 3.2	36	−9 °C step down
5	3.2 – 4.0	42	+6 °C step up

The initial reactor temperature is 30 °C. The profile includes both positive and negative step changes of varying magnitude, providing a rigorous test of the controller's ability to heat and cool the reactor through its available MVs.

5.4 Scenario Performance Comparison

Table 9: Validation Performance Across All Seven MV-Pairing Scenarios

#	Active MVs	Primary actuator	Ranking	Episode Reward
4	$T_{A,\text{in}}$ & F_B	Feed-A temperature	1st (best)	−100,206
7	T_{oil} & $T_{B,\text{in}}$	Oil bath temperature	2nd	−111,615
1	$T_{A,\text{in}}$ & $T_{B,\text{in}}$	Both feed temps	3rd	−139,519
2	T_{oil} & F_A	Oil bath temperature	4th	−165,048
6	T_{oil} & $T_{A,\text{in}}$	Oil bath temperature	5th	−198,516
5	$T_{B,\text{in}}$ & F_A	Feed-B temperature	6th	−209,715
3	T_{oil} & F_B	Oil bath temperature	7th (worst)	−216,590

The episode reward is the sum $\sum_{t=1}^{200} r_t = -100 \sum e_T^2$. A less negative value indicates tighter tracking of the stepped setpoint profile. Scenarios featuring $T_{A,\text{in}}$ as an active MV tend to perform better; this is consistent with the higher reactant concentration of Feed A ($C_{A,\text{in1}} = 1.0$ kmol/m^3 vs. 0.5), which amplifies its thermal influence on the reactor. Flow-rate MVs (F_A, F_B) are generally used as secondary actuators, with the primary temperature MV carrying most of the control burden.

5.5 Seven MV-Pairing Scenarios — Detailed Results

Each figure shows a two-panel plot: the top panel shows reactor temperature (solid blue) tracking the stepped setpoint (dashed red), with setpoint values annotated; the bottom panel shows the two active manipulated variables over time.

5.5.1 *Scenario 1:* $T_{A,\text{in}}$ *and* $T_{B,\text{in}}$

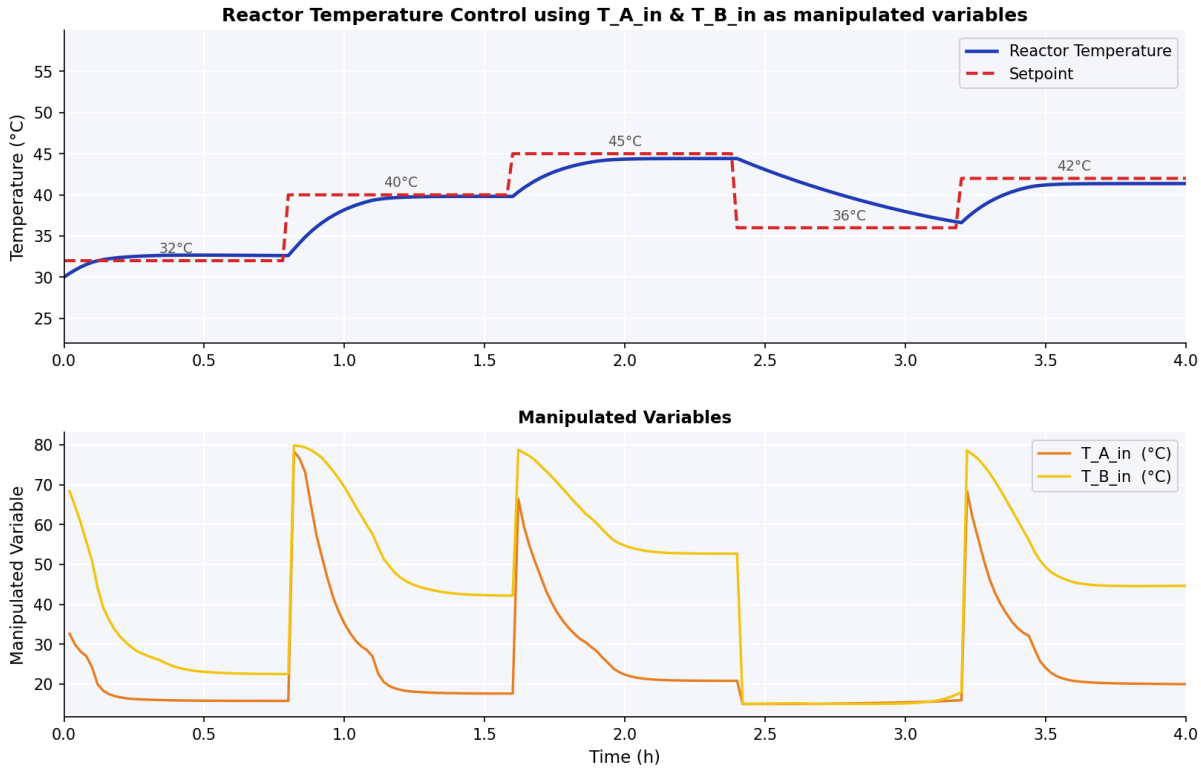

Figure 2: Scenario 1 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: $T_{A,\text{in}}$ (orange, °C) and $T_{B,\text{in}}$ (yellow, °C) manipulated variables. Episode reward: −139,519.

Observations: Both feed temperatures are used cooperatively. The agent drives both $T_{A,\text{in}}$ and $T_{B,\text{in}}$ to their maximum (80 °C) simultaneously at each upward step change, then reduces them in tandem when cooling is required. The near-identical trajectories of both MVs indicate the agent learned that applying them symmetrically is optimal — effectively treating them as a single high-authority heating actuator. Response to the large downward step at $t = 2.4$ h (−9 °C) is slower than upward steps, as cooling relies solely on reducing the convective heat input rather than active cooling.

5.5.2 *Scenario 2: T_{oil} and F_A*

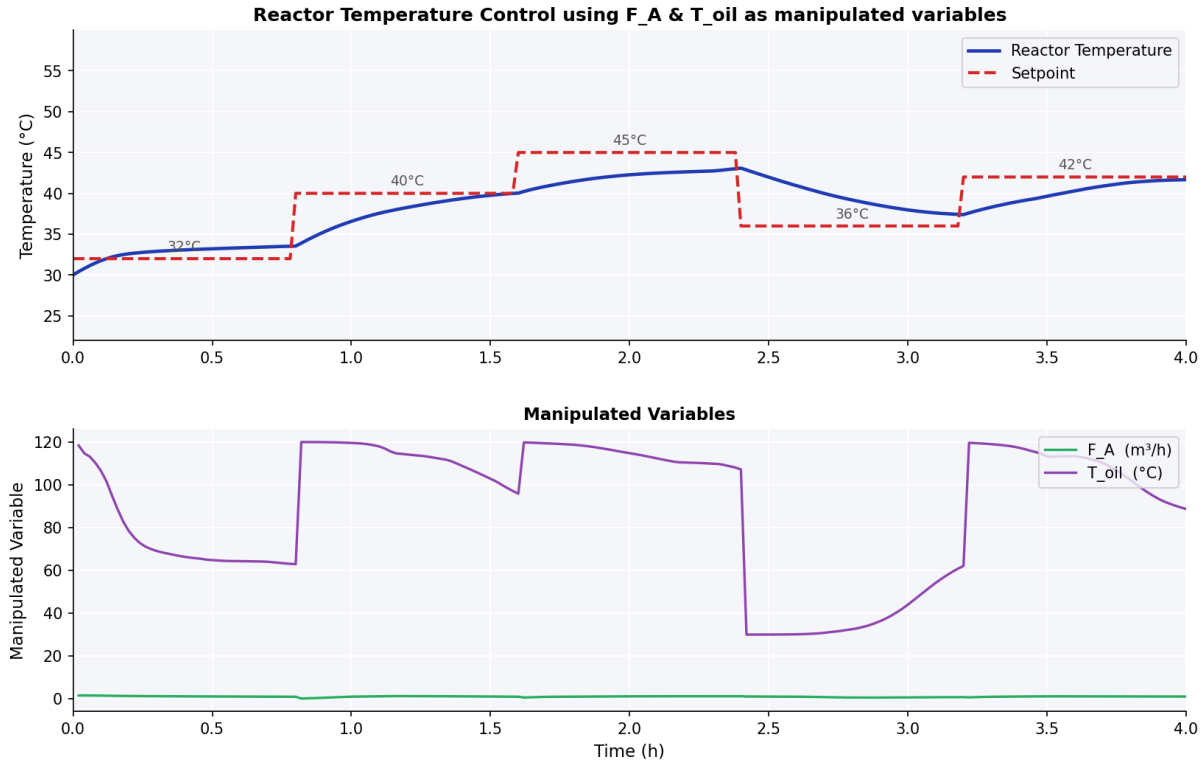

Figure 3: Scenario 2 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: F_A (green, m^3/h) and T_{oil} (purple, °C) manipulated variables. Episode reward: −165,048.

Observations: The oil bath temperature T_{oil} is the dominant actuator, sweeping the full range from 30 to 120 °C in response to each setpoint step. Feed-A flow rate F_A remains near zero throughout, indicating the agent determined that varying flow rate has negligible thermal authority compared to direct jacket heating. The oil bath's large temperature range (90 °C span) provides sufficient authority for moderate setpoint tracking, but the sluggish jacket heat-transfer dynamics ($UA/(\rho V C_p)$ timescale) result in slower response than direct feed temperature control, explaining the lower performance vs. Scenario 4.

5.5.3 Scenario 3: T_{oil} and F_B

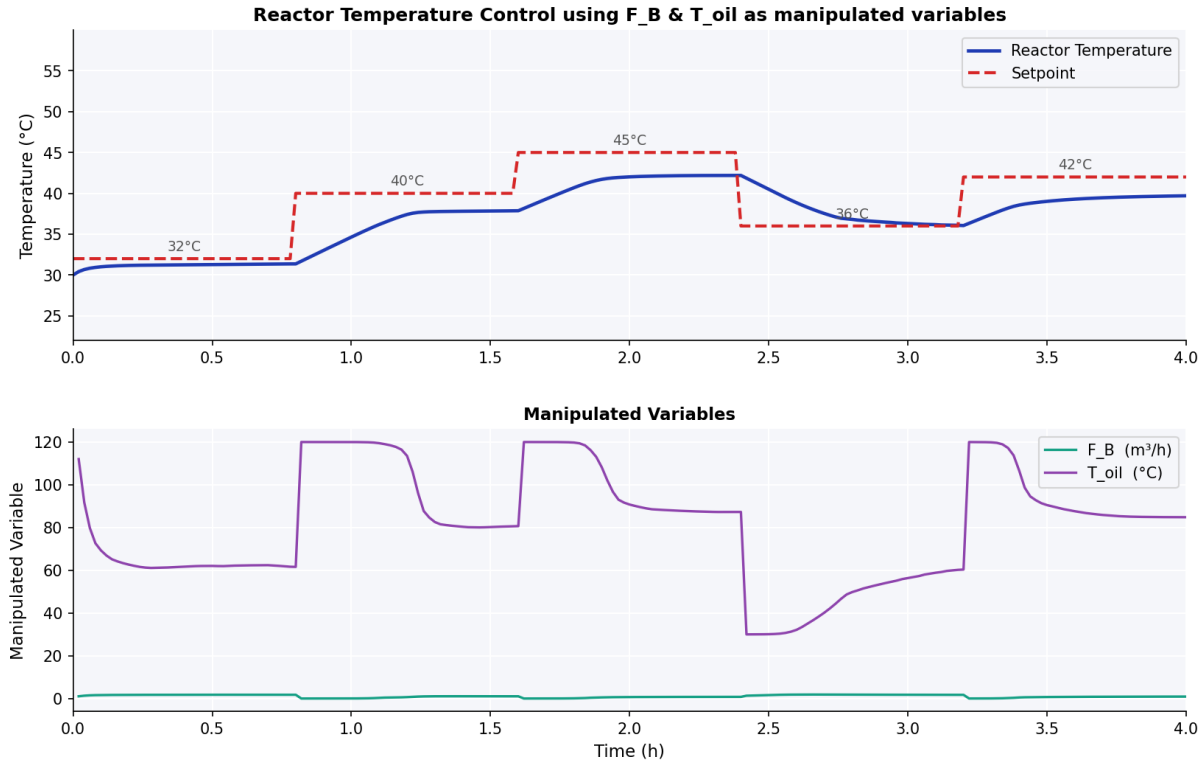

Figure 4: Scenario 3 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: F_B (green, m^3/h) and T_{oil} (purple, °C) manipulated variables. Episode reward: −216,590 (worst).

Observations: Scenario 3 is the worst-performing configuration. Like Scenario 2, T_{oil} dominates and F_B stays near its lower bound. The slightly worse performance vs. Scenario 2 is attributable to the lower reactant concentration of Feed B ($C_{A,\text{in2}} = 0.5$ vs. 1.0 kmol/m^3 for Feed A), which reduces F_B's indirect thermal influence via reaction heat. The oil bath alone cannot respond quickly enough to track the aggressive step changes, resulting in sustained tracking error during the transient periods following each step.

5.5.4 *Scenario 4: $T_{A,\text{in}}$ and F_B*

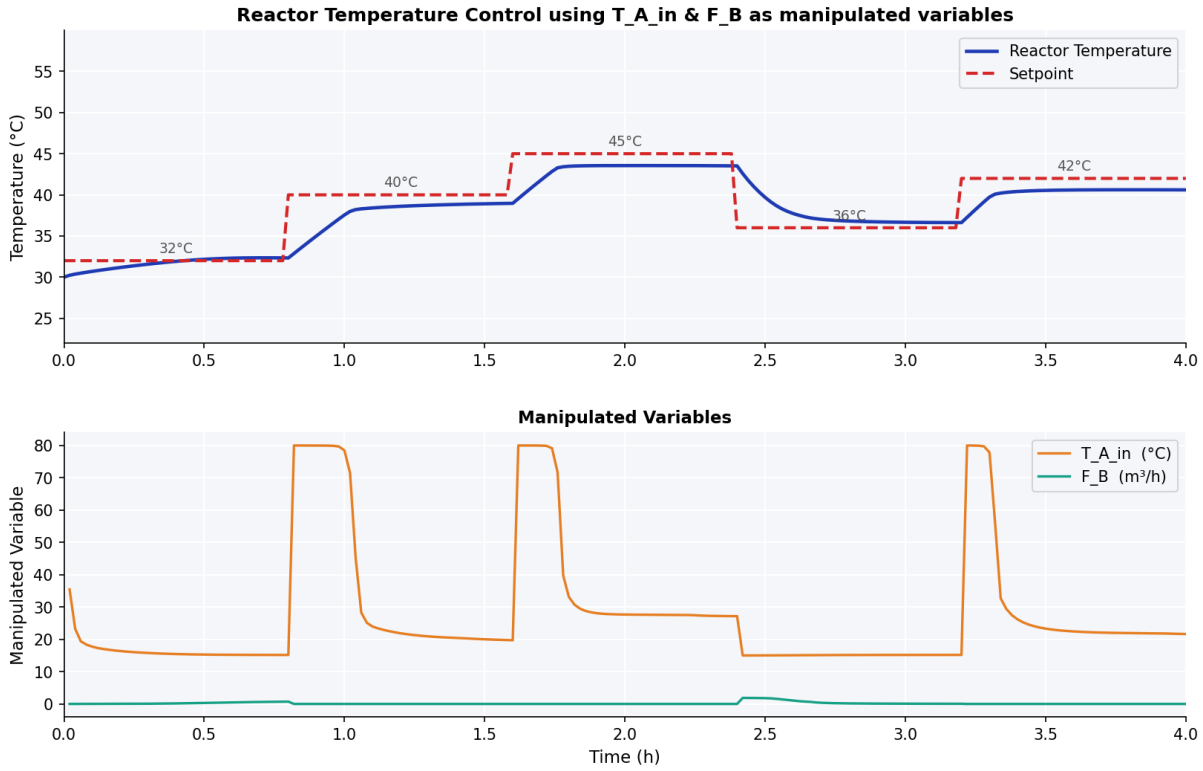

Figure 5: Scenario 4 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: $T_{A,\text{in}}$ (orange, °C) and F_B (teal, m^3/h) manipulated variables. Episode reward: −100,206 (best).

Observations: Scenario 4 achieves the best performance across all seven configurations. The agent drives $T_{A,\text{in}}$ aggressively, saturating at 80 °C during heating phases and dropping to near 15 °C during cooling. This provides a direct and fast thermal pathway: hot (or cold) feed enters the reactor every timestep, producing nearly immediate temperature change with no jacket thermal lag. F_B remains near zero, confirming that flow-rate manipulation adds marginal value when a high-authority temperature MV is available. The clean, responsive tracking visible across all five setpoint segments demonstrates that $T_{A,\text{in}}$ is the most effective individual thermal actuator in this model.

5.5.5 *Scenario 5: $T_{B,\text{in}}$ and F_A*

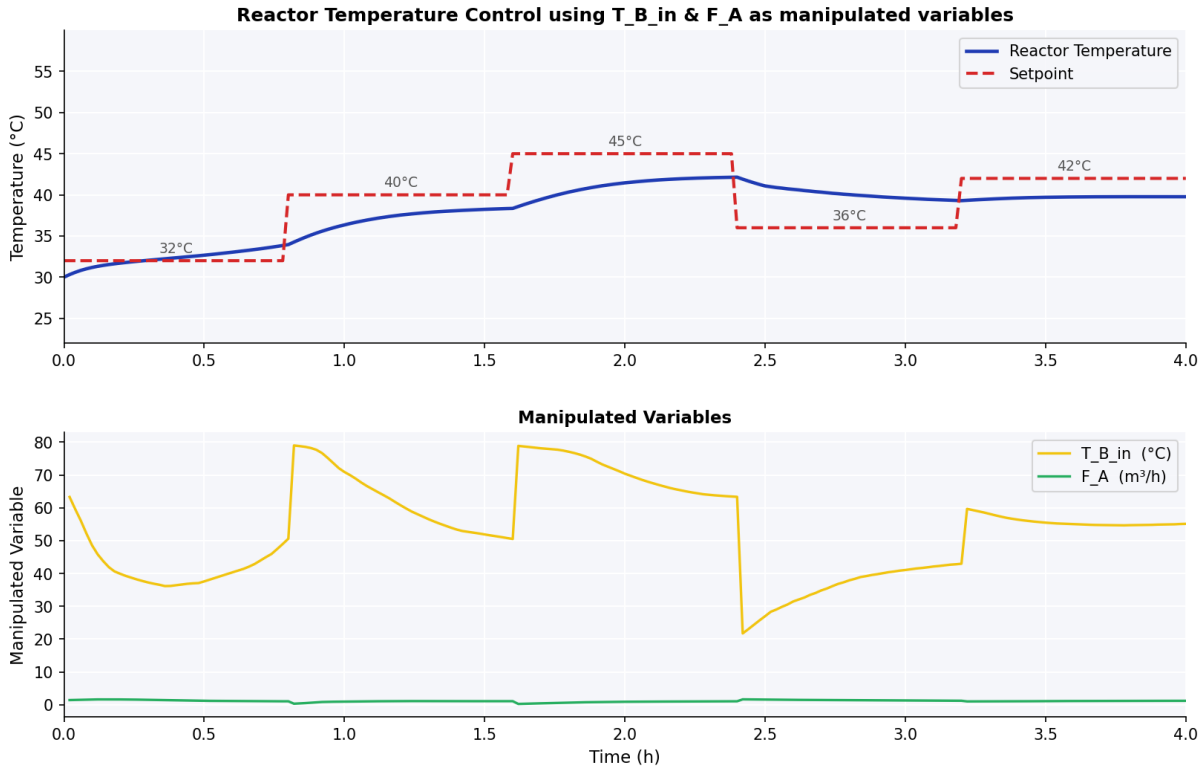

Figure 6: Scenario 5 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: $T_{B,\text{in}}$ (orange, °C) and F_A (green, m^3/h) manipulated variables. Episode reward: −209,715.

Observations: Scenario 5 is the mirror of Scenario 4 with Feed B temperature instead of Feed A. Despite using the same pairing structure (feed temperature + flow rate), performance is significantly worse (−209,715 vs. −100,206). This is entirely consistent with the Feed A/B asymmetry: Feed A carries twice the reactant concentration (1.0 vs. 0.5 kmol/m^3), so manipulating $T_{A,\text{in}}$ has both a direct convective thermal effect *and* an indirect effect via the reaction rate at the higher concentration. Feed B temperature lacks this amplification. F_A again remains near its minimum, confirming that flow-rate manipulation is subordinate to feed temperature control.

5.5.6 *Scenario 6:* T_{oil} *and* $T_{A,\text{in}}$

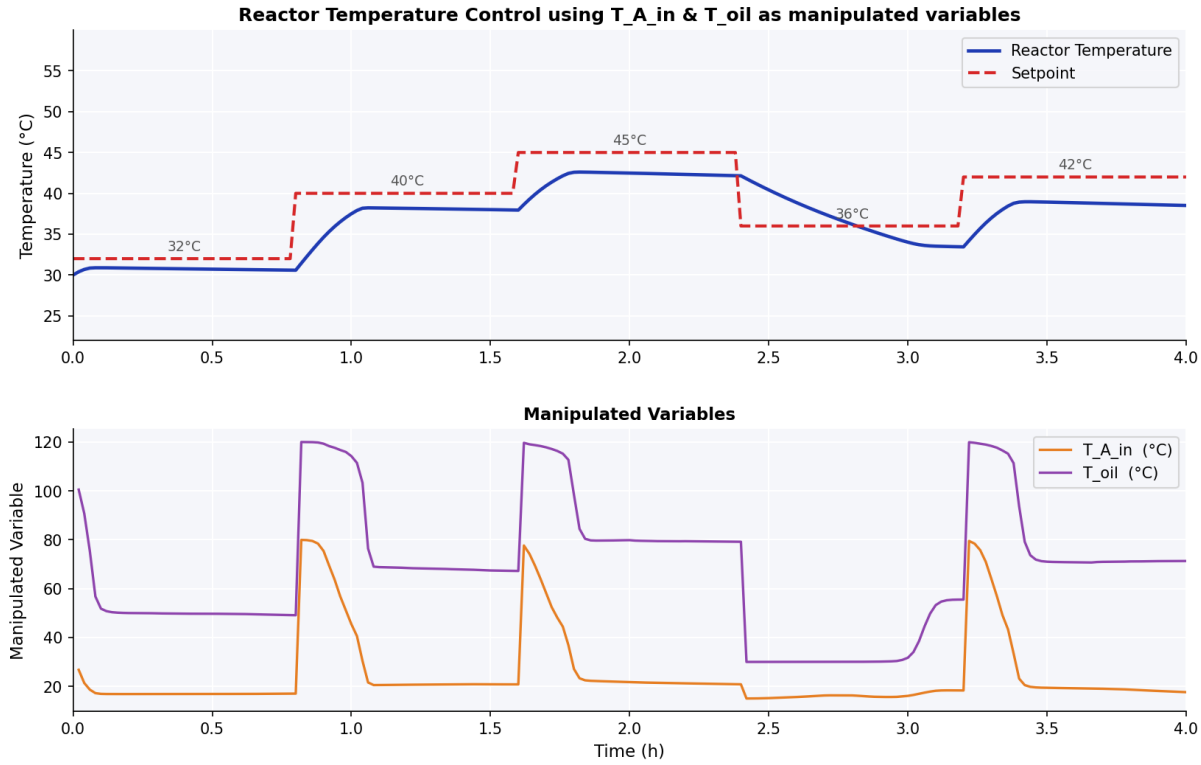

Figure 7: Scenario 6 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: $T_{A,\text{in}}$ (orange, °C) and T_{oil} (purple, °C) manipulated variables. Episode reward: −198,516.

Observations: Scenario 6 pairs the oil bath (slow, high-authority jacket actuator) with Feed A temperature (fast, direct convective actuator). The oil bath makes large, coarse step changes in response to each setpoint transition, while $T_{A,\text{in}}$ provides faster supplementary adjustment. Despite having $T_{A,\text{in}}$ available, the combination performs worse than Scenario 4 (where $T_{A,\text{in}}$ alone dominates). This suggests the oil bath's presence as a competing actuator creates a control allocation challenge — the agent must split authority between two MVs with very different response timescales, leading to suboptimal coordination.

5.5.7 *Scenario 7:* T_{oil} *and* $T_{B,\text{in}}$

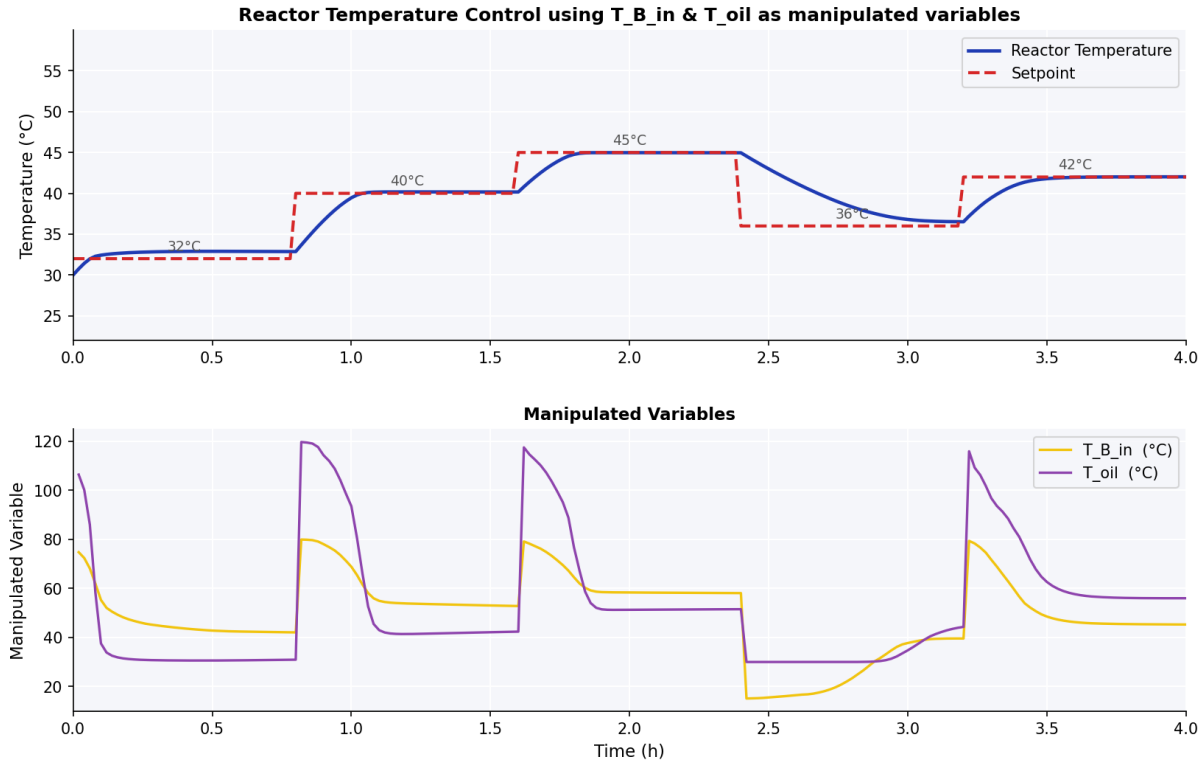

Figure 8: Scenario 7 — Top: Reactor temperature (°C) vs. time (h) tracking the stepped setpoint. Bottom: $T_{B,\text{in}}$ (orange, °C) and T_{oil} (purple, °C) manipulated variables. Episode reward: −111,615 (2nd best).

Observations: Scenario 7 is the second-best performer. T_{oil} makes the large primary control moves, while $T_{B,\text{in}}$ provides supplementary fine adjustment. The oil bath's large operating range (30–120 °C) compensates for the lower thermal authority of Feed B. The combination works better here than in Scenarios 2, 3, and 6 because $T_{B,\text{in}}$ can respond immediately (within one timestep) to setpoint changes while the jacket heat transfer catches up, effectively providing fast-response fine tuning on top of the oil bath's high steady-state authority.

5.6 All-Scenarios Summary Panel

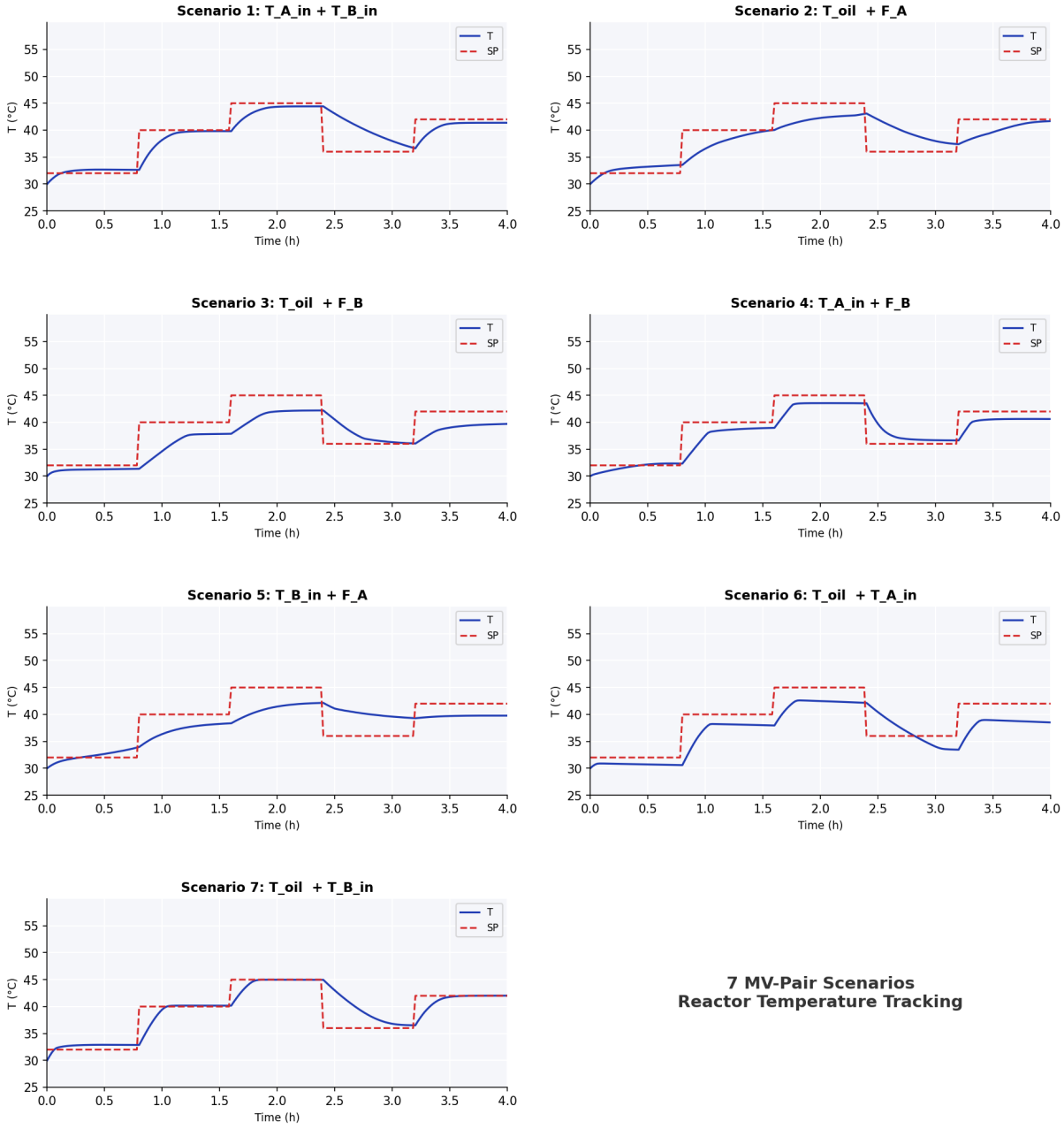

Figure 9: Composite summary: reactor temperature tracking (solid blue) against the stepped setpoint profile (dashed red) across all seven MV-pairing scenarios. All scenarios were validated under identical initial conditions ($T_0 = 30$ °C) and the same setpoint profile ($32 \rightarrow 40 \rightarrow 45 \rightarrow 36 \rightarrow 42$ °C every 0.8 h). The universal agent adapts its control strategy to the available MVs in each scenario without retraining.

The composite panel reveals a consistent qualitative pattern across all scenarios: temperature rises are generally faster and more precise than temperature decreases, which is physically expected since most MVs (hot feed temperatures, oil bath heating) provide heating rather than cooling authority. Cooling must rely on reducing heating input and allowing reactor dynamics to dissipate heat naturally, which is inherently slower.

6 Discussion

6.1 Control Authority Analysis

The performance ranking in Table 9 reveals a clear hierarchy of MV effectiveness for temperature control in this system:

1. **Feed temperature MVs ($T_{A,\text{in}}$, $T_{B,\text{in}}$):** Highest authority. Direct convective heat transfer with no lag; effect felt within one Euler sub-step. $T_{A,\text{in}}$ is more effective than $T_{B,\text{in}}$ due to Feed A's higher reactant concentration amplifying the indirect reaction-heat pathway.
2. **Oil bath temperature (T_{oil}):** High steady-state authority but slower response due to jacket heat-transfer dynamics (governed by $UA/(\rho V C_p)$). Effective for maintaining setpoints but disadvantaged during rapid step changes.
3. **Flow rates (F_A, F_B):** Lowest thermal authority in isolation. Changing flow rate affects residence time and dilution, which influences temperature indirectly via reactant concentration. The agent consistently deprioritises flow-rate MVs when a temperature MV is available.

6.2 Universal Agent Behaviour

The single universal policy network successfully learns to condition its control strategy on the active MV pair (communicated via the 5-bit scenario mask in the state vector). Without any explicit programming of scenario-specific logic, the agent:

- Saturates the highest-authority available MV (feed temperature) to its physical limit immediately upon a setpoint step change.
- Uses secondary low-authority MVs (flow rates) sparingly or not at all when a temperature MV is available.
- Reduces the primary MV once the reactor approaches the new setpoint to avoid overshoot, reflecting the integral-derivative awareness embedded in the state vector.

6.3 Limitations and Future Work

- **Conservative transient response:** The quadratic reward $r_t = -100\, e_T^2$ penalises overshoot and undershoot equally. As a result, the agent learns conservative control that avoids overshooting, at the cost of slower rise times. Reward shaping (e.g., asymmetric penalties, or a bonus for reaching the setpoint band within a time limit) could produce more aggressive, oscillatory control.
- **Training horizon:** 10,000 episodes is sufficient for qualitative convergence but further training with a higher update-to-data ratio (multiple gradient updates per environment step) would improve steady-state accuracy.
- **Flow-rate utilisation:** In all scenarios pairing a flow-rate MV with a temperature

MV, the flow rate is largely unused. A reward term penalising excessive reliance on a single MV would encourage more balanced use of both available actuators.

- **Process noise:** The current model uses deterministic dynamics. Adding Gaussian measurement noise and process disturbances would improve robustness and better reflect real laboratory conditions.

7 Conclusion

This report has presented the design, implementation, and validation of a TD3-based deep reinforcement learning controller for a jacketed CSTR with a full energy balance. The key findings are:

- A **single universal agent** with a 13-D state vector (including a 5-bit scenario mask) and a 5-D continuous action space successfully learns effective reactor-temperature control across all seven MV-pairing scenarios without per-scenario retraining. This demonstrates the feasibility of scenario-conditioned universal DRL controllers for process systems.
- The **energy balance** introduced in this work — combining convective feed enthalpy, exothermic reaction heat, and jacket heat transfer from the hot oil bath — transforms reactor temperature from a direct manipulated input into a physically modelled state variable, substantially improving simulation realism.
- **Control performance** varies significantly across scenarios. Scenarios with $T_{A,\text{in}}$ as an active MV achieve the best results (best: Scenario 4, reward = −100,206); scenarios relying solely on T_{oil} with a flow-rate secondary MV perform worst (Scenario 3, reward = −216,590). The ranking is consistent with the physical thermal authority analysis: direct feed temperature control provides the fastest response, while oil bath control suffers from jacket lag.
- **Implicit MV prioritisation** emerges from training without explicit programming: the agent consistently assigns primary control responsibility to the highest-authority available MV (feed temperature > oil bath temperature > flow rate) — a result consistent with process engineering intuition.
- **PSO-calibrated kinetics** ($k_0 = 3.0 \times 10^8$ h^{-1}, $E_a/R = 7500$ K) produce physically consistent temperature dynamics at the operating range of 30–50 °C, validating the first-principles model against laboratory CSTR data.
- The **Streamlit dashboard** and modular Python codebase provide a complete interactive platform for exploring all seven scenarios in real-time, enabling rapid visual comparison of controller behaviour across MV pairings.

References

[1] Martinez, B., Rodriguez, M., & Diaz, I. (2022). CSTR control with deep reinforcement learning. *Proceedings of the 14th International Symposium on Process Systems Engineering (PSE 2021+)*, Kyoto, Japan. https://doi.org/10.1016/B978-0-323-85159-6.50282-7

[2] Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods. *Proc. 35th International Conference on Machine Learning (ICML 2018)*.

[3] Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[4] Shin, J., Badgwell, T. A., Liu, K., & Lee, J. H. (2019). Reinforcement Learning — Overview of recent progress and implications for process control. *Computers and Chemical Engineering*, 127, 282–294.

[5] PSO Parameter Estimation — MLAPC Lab CSTR. *PSO_CSTR_MLAPClab.docx*. UICPS FISAC Internal Report, 2026.

[6] Spinning Up in Deep RL (2021). OpenAI. https://spinningup.openai.com

[7] ITarasu (2026). *ITarasu — Process Control and AI Integration Platform*. https://itarasu.com/